OOP REPORT

Hiba Amanulla S23108227 - Ghala Alghamdi S23108212

CODE

MAIN CLASS

The Main class implements an ATM system with a graphical user interface (GUI). It initializes the main frame, loads user data, and can do operations login, account functionality, and admin panel. It has functionalities for creating and managing user accounts, including options for account creation, PIN management, and transaction logging, while ensuring input validation and facilitating user logout.

```
import java.awt.*;
import java.util.ArrayList;
import java.util.List;
import javax.swing.*;

public class Main {
    private JFrame frame;
    private JPanel mainPanel;
    private CardLayout cardLayout;
    private User currentUser;
    private List<User> users;
    private FileManager fileManager;

     // Constructor to initialize the ATM System
    public Main() {
        frame = new JFrame("ATM System");
        cardLayout = new CardLayout();
        mainPanel = new JPanel(cardLayout);
        users = new ArrayList<>();
        fileManager = new FileManager();
        users = fileManager.loadUsers(); // Load users from file

        mainPanel.add(createLoginPage(), "Login");
        mainPanel.add(createFunctionalityPage(), "Functionality");
        mainPanel.add(createAdminPanel(), "Admin Panel");

        frame.add(mainPanel);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack();
        frame.setVisible(true);
    }
```

```java
    // Function to create the login page
    private JPanel createLoginPage() {
        JPanel panel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 10, 10, 10);

        // Create input fields and buttons for login
        JTextField userIDField = new JTextField(20);
        JPasswordField pinField = new JPasswordField(20);
        JButton loginButton = new JButton("Login");
        JButton registerButton = new JButton("Register");

        // Add labels and fields to the panel
        gbc.gridx = 0; gbc.gridy = 0; panel.add(new JLabel("User ID:"), gbc);
        gbc.gridx = 1; panel.add(userIDField, gbc);
        gbc.gridx = 0; gbc.gridy = 1; panel.add(new JLabel("PIN:"), gbc);
        gbc.gridx = 1; panel.add(pinField, gbc);
        gbc.gridx = 0; gbc.gridy = 2; gbc.gridwidth = 2; panel.add(loginButton, gbc);
        gbc.gridy = 3; panel.add(registerButton, gbc);

        // Set action listeners for buttons
        loginButton.addActionListener(e -> login(userIDField.getText(), new
String(pinField.getPassword())));
        registerButton.addActionListener(e -> registerUser());
        return panel;
    }

    // Function to create the functionality page after login
    private JPanel createFunctionalityPage() {
        JPanel panel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 10, 10, 10);

        // Components for account selection and balance display
        JComboBox<String> accountComboBox = new JComboBox<>();
        JLabel balanceLabel = new JLabel("Balance: N/A");

        // Add labels and fields to the panel
        gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;
        panel.add(new JLabel("Select Account:"), gbc);
        gbc.gridy = 1;
        panel.add(accountComboBox, gbc);
```

```java
        gbc.gridy = 2;
        panel.add(balanceLabel, gbc);

        // Action listener for account selection
        accountComboBox.addActionListener(e -> {
            String selectedAccountNumber = (String) accountComboBox.getSelectedItem();
            Account selectedAccount = getAccountByNumber(selectedAccountNumber);
            if (selectedAccount != null) {
                balanceLabel.setText("Balance: " + selectedAccount.getBalance());
            } else {
                balanceLabel.setText("Balance: N/A");
            }
        });

        // Create buttons for various functionalities
        JButton depositButton = new JButton("Deposit");
        JButton withdrawButton = new JButton("Withdraw");
        JButton transferButton = new JButton("Transfer");
        JButton logoutButton = new JButton("Log Out");
        JButton adminPanelButton = new JButton("Admin Panel");

        // Add buttons to the panel
        gbc.gridy = 3; panel.add(depositButton, gbc);
        gbc.gridy = 4; panel.add(withdrawButton, gbc);
        gbc.gridy = 5; panel.add(transferButton, gbc);
        gbc.gridy = 6; panel.add(logoutButton, gbc);
        gbc.gridy = 7; panel.add(adminPanelButton, gbc);

        // Set action listeners for buttons
        depositButton.addActionListener(e -> deposit((String)
accountComboBox.getSelectedItem(), balanceLabel));
        withdrawButton.addActionListener(e -> withdraw((String)
accountComboBox.getSelectedItem(), balanceLabel));
        transferButton.addActionListener(e -> transfer((String)
accountComboBox.getSelectedItem(), balanceLabel));
        logoutButton.addActionListener(e -> logout());
        adminPanelButton.addActionListener(e -> cardLayout.show(mainPanel, "Admin Panel"));

        return panel;
    }

    // Function to create the admin panel for admin functionalities
    private JPanel createAdminPanel() {
        JPanel panel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
```

```java
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 10, 10, 10);

        // Create buttons for admin functionalities
        JButton createAccountButton = new JButton("Create New Account");
        JButton accountManagementButton = new JButton("Account Management");
        JButton generateReportsButton = new JButton("Generate Reports");
        JButton backButton = new JButton("Back");

        // Add buttons to the panel
        gbc.gridx = 0; gbc.gridy = 0; panel.add(createAccountButton, gbc);
        gbc.gridy = 1; panel.add(accountManagementButton, gbc);
        gbc.gridy = 2; panel.add(generateReportsButton, gbc);
        gbc.gridy = 3; panel.add(backButton, gbc);

        // Set action listeners for buttons
        createAccountButton.addActionListener(e -> createNewAccount());
        accountManagementButton.addActionListener(e -> manageAccountsByPin());
        generateReportsButton.addActionListener(e -> {
            if (currentUser != null) {
                showUserReport(currentUser); // Show report for the current user
            } else {
                JOptionPane.showMessageDialog(frame, "No user is currently logged in.");
            }
        });
        backButton.addActionListener(e -> cardLayout.show(mainPanel, "Functionality"));

        return panel;
    }

    // Function to display the user report
    private void showUserReport(User user) {
        StringBuilder report = new StringBuilder();

        report.append("User Report for ").append(user.getName()).append(" 
(").append(user.getUserID()).append("):\n");

        // Adding account details
        for (Account account : user.getAccounts()) {
            report.append("Account Number: ").append(account.getAccountNumber())
                  .append(", Balance: ").append(account.getBalance()).append("\n");
        }

        report.append("Transaction History:\n");
```

```java
        // Assuming user has a method to get transaction history
        for (Transaction transaction : user.getTransactionHistory()) {
            report.append(transaction.toString()).append("\n");
        }

        // Display the report in a dialog
        JOptionPane.showMessageDialog(frame, report.toString(), "User Report",
JOptionPane.INFORMATION_MESSAGE);
    }

    // Function to handle user login
    private void login(String userID, String pin) {
        for (User user : users) {
            if (user.getUserID().equals(userID) && user.getPin().equals(pin)) {
                currentUser = user;
                updateAccountComboBox((JComboBox<String>) ((JPanel)
mainPanel.getComponent(1)).getComponent(1));
                cardLayout.show(mainPanel, "Functionality");
                return;
            }
        }
        JOptionPane.showMessageDialog(frame, "Invalid User ID or PIN.");
    }

    // Function to update account selection combo box
    private void updateAccountComboBox(JComboBox<String> accountComboBox) {
        accountComboBox.removeAllItems(); // Clear existing items
        for (Account account : currentUser.getAccounts()) {
            accountComboBox.addItem(account.getAccountNumber()); // Add account numbers to the
combo box
        }
    }

    // Function to register a new user
    private void registerUser() {
        JPanel panel = new JPanel(new GridLayout(0, 1));
        JTextField nameField = new JTextField(20);
        JTextField userIDField = new JTextField(20);
        JPasswordField pinField = new JPasswordField(20);
        JTextField initialBalanceField = new JTextField(20);
        String[] accountTypes = {"Checking Account", "Savings Account"};
        JComboBox<String> accountTypeComboBox = new JComboBox<>(accountTypes);

        // Add fields to the panel
        panel.add(new JLabel("Enter your name:"));
```

```java
        panel.add(nameField);
        panel.add(new JLabel("Enter a new User ID:"));
        panel.add(userIDField);
        panel.add(new JLabel("Enter your PIN:"));
        panel.add(pinField);
        panel.add(new JLabel("Select Account Type:"));
        panel.add(accountTypeComboBox);
        panel.add(new JLabel("Enter Initial Balance:"));
        panel.add(initialBalanceField);

        int result = JOptionPane.showConfirmDialog(frame, panel, "Register",
                JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);

        if (result == JOptionPane.OK_OPTION) {
            String name = nameField.getText().trim();
            String userID = userIDField.getText().trim();
            String pin = new String(pinField.getPassword()).trim();
            String accountType = (String) accountTypeComboBox.getSelectedItem();
            String initialBalanceString = initialBalanceField.getText().trim();

            // Validate input fields
            if (name.isEmpty() || userID.isEmpty() || pin.isEmpty() ||
initialBalanceString.isEmpty()) {
                JOptionPane.showMessageDialog(frame, "All fields must be filled.");
                return;
            }

            double initialBalance;
            try {
                initialBalance = Double.parseDouble(initialBalanceString);
                if (initialBalance < 0) {
                    JOptionPane.showMessageDialog(frame, "Initial balance must be a positive
number.");
                    return;
                }
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(frame, "Initial balance must be a valid
number.");
                return;
            }

            // Check for existing user ID
            for (User user : users) {
                if (user.getUserID().equals(userID)) {
```

```
                    JOptionPane.showMessageDialog(frame, "User ID already exists. Please
choose another.");
                    return;
                }
            }

            // Create new account and user
            String accountNumber = (accountType.equals("Checking Account") ? "C-ACC" : "S-
ACC") + System.currentTimeMillis();
            Account newAccount = createAccount(accountType, accountNumber, initialBalance);
            User newUser = new User(name, userID, pin);
            newUser.addAccount(newAccount);
            users.add(newUser);
            fileManager.saveUsers(users);
            JOptionPane.showMessageDialog(frame, "Registration successful! You can now log
in.");
        }
    }

    // Function to create a new account based on type
    private Account createAccount(String accountType, String accountNumber, double
initialBalance) {
        if (accountType.equals("Checking Account")) {
            return new CheckingAccount(accountNumber, initialBalance, 100);
        } else {
            return new SavingsAccount(accountNumber, initialBalance, 1.5);
        }
    }

    // Function to handle deposits
    private void deposit(String accountNumber, JLabel balanceLabel) {
        Account account = getAccountByNumber(accountNumber);
        if (account != null) {
            double amount = getValidatedAmount("Enter amount to deposit into account " +
accountNumber + ":");
            if (isValidAmount(amount)) {
                boolean success = account.deposit(amount, currentUser, fileManager);
                if (success) {
                    logTransaction("Deposit", amount, accountNumber);
                    JOptionPane.showMessageDialog(frame, "Deposit successful! New balance: " +
account.getBalance());
                    balanceLabel.setText("Balance: " + account.getBalance());
                } else {
                    JOptionPane.showMessageDialog(frame, "Deposit failed. Please try again.");
                }
```

```
            }
        } else {
            JOptionPane.showMessageDialog(frame, "No accounts available for deposit.");
        }
    }

    // Function to handle withdrawals
    private void withdraw(String accountNumber, JLabel balanceLabel) {
        Account account = getAccountByNumber(accountNumber);
        if (account != null) {
            double amount = getValidatedAmount("Enter amount to withdraw from account " +
accountNumber + ":");
            if (isValidAmount(amount) && amount <= account.getBalance()) {
                boolean success = account.withdraw(amount, currentUser, fileManager);
                if (success) {
                    logTransaction("Withdrawal", amount, accountNumber);
                    JOptionPane.showMessageDialog(frame, "Withdrawal successful! New balance:
" + account.getBalance());
                    balanceLabel.setText("Balance: " + account.getBalance());
                } else {
                    JOptionPane.showMessageDialog(frame, "Withdrawal failed. Please try
again.");
                }
            } else {
                JOptionPane.showMessageDialog(frame, "Invalid withdrawal amount.");
            }
        } else {
            JOptionPane.showMessageDialog(frame, "No accounts available for withdrawal.");
        }
    }

    // Function to handle transfers between accounts
    private void transfer(String accountNumber, JLabel balanceLabel) {
        Account account = getAccountByNumber(accountNumber);
        if (currentUser != null && currentUser.getAccounts().size() >= 2) {
            DefaultListModel<String> listModel = new DefaultListModel<>();
            for (Account acc : currentUser.getAccounts()) {
                if (!acc.getAccountNumber().equals(accountNumber)) {
                    listModel.addElement(acc.getAccountNumber());
                }
            }

            JList<String> accountList = new JList<>(listModel);
            accountList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
            JScrollPane scrollPane = new JScrollPane(accountList);
```

```
            double amount = getValidatedAmount("Enter amount to transfer from account " +
accountNumber + ":");
            if (isValidAmount(amount) && amount <= account.getBalance()) {
                int result = JOptionPane.showConfirmDialog(frame, scrollPane, "Select Target
Account", JOptionPane.OK_CANCEL_OPTION);
                if (result == JOptionPane.OK_OPTION) {
                    String selectedAccountNumber = accountList.getSelectedValue();
                    if (selectedAccountNumber != null) {
                        Account targetAccount = getAccountByNumber(selectedAccountNumber);
                        boolean withdrawalSuccess = account.withdraw(amount, currentUser,
fileManager);
                        boolean depositSuccess = targetAccount.deposit(amount, currentUser,
fileManager);

                        if (withdrawalSuccess && depositSuccess) {
                            JOptionPane.showMessageDialog(frame, "Transfer successful!");
                            logTransaction("Transfer Out", amount, accountNumber);
                            logTransaction("Transfer In", amount, selectedAccountNumber);
                        } else {
                            JOptionPane.showMessageDialog(frame, "Transfer failed. Please try
again.");
                        }
                    } else {
                        JOptionPane.showMessageDialog(frame, "No target account selected.");
                    }
                }
            } else {
                JOptionPane.showMessageDialog(frame, "Invalid transfer amount.");
            }
        } else {
            JOptionPane.showMessageDialog(frame, "You need at least two accounts to
transfer.");
        }
    }

    // Function to create a new account
    private void createNewAccount() {
        JPanel panel = new JPanel(new GridLayout(0, 1));

        JRadioButton checkingButton = new JRadioButton("Checking Account");
        JRadioButton savingsButton = new JRadioButton("Savings Account");
        ButtonGroup accountTypeGroup = new ButtonGroup();
        accountTypeGroup.add(checkingButton);
        accountTypeGroup.add(savingsButton);
```

```java
        panel.add(new JLabel("Select Account Type:"));
        panel.add(checkingButton);
        panel.add(savingsButton);

        JTextField initialBalanceField = new JTextField();
        panel.add(new JLabel("Enter Initial Balance:"));
        panel.add(initialBalanceField);

        int result = JOptionPane.showConfirmDialog(frame, panel, "Create New Account",
JOptionPane.OK_CANCEL_OPTION);

        if (result == JOptionPane.OK_OPTION) {
            String accountType = checkingButton.isSelected() ? "Checking Account" : "Savings
Account";
            String accountNumber = (accountType.equals("Checking Account") ? "C-ACC" : "S-
ACC") + System.currentTimeMillis();

            String initialBalanceString = initialBalanceField.getText().trim();
            if (initialBalanceString.isEmpty()) {
                JOptionPane.showMessageDialog(frame, "Initial balance cannot be empty.");
                return;
            }

            double initialBalance;
            try {
                initialBalance = Double.parseDouble(initialBalanceString);
                if (initialBalance < 0) {
                    JOptionPane.showMessageDialog(frame, "Initial balance must be a positive
number.");
                    return;
                }
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(frame, "Invalid amount. Please enter a numeric
value.");
                return;
            }

            Account newAccount = createAccount(accountType, accountNumber, initialBalance);
            currentUser.addAccount(newAccount);
            fileManager.saveUsers(users);
            JOptionPane.showMessageDialog(frame, "New account created! Account Number: " +
accountNumber);
        }
    }
```

```java
    // Function to manage accounts by PIN
    private void manageAccountsByPin() {
        JPanel panel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 10, 10, 10);
        JPasswordField pinField = new JPasswordField(20);

        gbc.gridx = 0; gbc.gridy = 0;
        panel.add(new JLabel("Enter User PIN:"), gbc);
        gbc.gridx = 1; panel.add(pinField, gbc);

        if (currentUser != null) {
            gbc.gridx = 0; gbc.gridy = 1;
            panel.add(new JLabel("Account Name: " + currentUser.getName()), gbc);
            gbc.gridy = 2;
            panel.add(new JLabel("User ID: " + currentUser.getUserID()), gbc);
        }

        int result = JOptionPane.showConfirmDialog(frame, panel, "Account Management",
JOptionPane.OK_CANCEL_OPTION);

        if (result == JOptionPane.OK_OPTION) {
            String pin = new String(pinField.getPassword()).trim();
            if (currentUser.getPin().equals(pin)) {
                String[] options = {"Change PIN", "Unlock Account"};
                int choice = JOptionPane.showOptionDialog(frame, "Select an action:", "Account
Management",
                        JOptionPane.DEFAULT_OPTION, JOptionPane.QUESTION_MESSAGE, null,
options, options[0]);

                if (choice == -1) return; // User closed the dialog

                if (choice == 0) { // Change PIN
                    JPanel newPinPanel = new JPanel(new GridBagLayout());
                    JPasswordField newPinField = new JPasswordField(20);
                    JPasswordField confirmPinField = new JPasswordField(20);

                    gbc.gridx = 0; gbc.gridy = 0;
                    newPinPanel.add(new JLabel("Enter New PIN:"), gbc);
                    gbc.gridx = 1; newPinPanel.add(newPinField, gbc);

                    gbc.gridx = 0; gbc.gridy = 1;
                    newPinPanel.add(new JLabel("Confirm New PIN:"), gbc);
```

```
                    gbc.gridx = 1; newPinPanel.add(confirmPinField, gbc);

                    int newPinResult = JOptionPane.showConfirmDialog(frame, newPinPanel,
"Change PIN", JOptionPane.OK_CANCEL_OPTION);
                    if (newPinResult == JOptionPane.OK_OPTION) {
                        String newPin = new String(newPinField.getPassword()).trim();
                        String confirmPin = new String(confirmPinField.getPassword()).trim();

                        if (newPin.equals(confirmPin)) {
                            fileManager.updateUserPassword(currentUser.getUserID(), newPin);
                            currentUser.setPin(newPin);
                            JOptionPane.showMessageDialog(frame, "PIN updated successfully!");
                        } else {
                            JOptionPane.showMessageDialog(frame, "New PINs do not match.
Please try again.");
                        }
                    }
                } else if (choice == 1) { // Unlock Account
                    // Placeholder for unlock account logic
                    JOptionPane.showMessageDialog(frame, "Account unlocked successfully!");
                }
            } else {
                JOptionPane.showMessageDialog(frame, "Incorrect PIN.");
            }
        }
    }

    // Function to log out the current user
    private void logout() {
        currentUser = null;
        cardLayout.show(mainPanel, "Login");
    }

    // Function to get an account by its number from the current user
    private Account getAccountByNumber(String accountNumber) {
        if (currentUser != null) {
            for (Account account : currentUser.getAccounts()) {
                if (account.getAccountNumber().equals(accountNumber)) {
                    return account;
                }
            }
        }
        return null;
    }
```

```java
    // Function to validate and get amount input
    private double getValidatedAmount(String message) {
        String input = JOptionPane.showInputDialog(frame, message);
        if (input != null) {
            try {
                return Double.parseDouble(input);
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(frame, "Invalid amount. Please enter a numeric
value.");
            }
        }
        return 0;
    }

    // Function to check if the amount is valid (not negative)
    private boolean isValidAmount(double amount) {
        return amount > 0;
    }

    // Function to log transactions
    private void logTransaction(String type, double amount, String accountNumber) {
        if (currentUser != null) {
            Transaction transaction = new Transaction(currentUser.getUserID(), accountNumber,
type, amount);
            currentUser.addTransaction(transaction); // Assuming User class has this method
        }
    }

    // Main method to run the ATM application
    public static void main(String[] args) {
        SwingUtilities.invokeLater(Main::new);
    }
}
```

ACCOUNTS CLASS

The Account class serves as an abstract base for different types of bank accounts, encapsulating common functionalities such as account initialization, deposit and withdrawal operations, and transaction logging.

```java
import java.util.ArrayList;
import java.util.List;

public abstract class Account {
```

```java
    protected String accountNumber;
    protected double balance;
    protected List<Transaction> transactionHistory;

    // Constructor to initialize the account with a number and initial balance
    public Account(String accountNumber, double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative.");
        }
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.transactionHistory = new ArrayList<>();
    }

    // Deposit money into the account
    // Validates the deposit amount, updates the balance, logs the transaction, and updates
file storage
    public boolean deposit(double amount, User user, FileManager fileManager) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive.");
        }

        balance += amount;
        logTransaction("Deposit", amount, user);

        fileManager.logTransaction(new Transaction(user.getUserID(), accountNumber, "Deposit",
amount));
        fileManager.updateAccountBalance(user.getUserID(), accountNumber, balance);

        return true;
    }

    // Withdraw money from the account
    // Validates the withdrawal amount, checks for sufficient funds, updates the balance, and
logs the transaction
    public boolean withdraw(double amount, User user, FileManager fileManager) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive.");
        }
        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient funds for withdrawal.");
        }

        balance -= amount;
        logTransaction("Withdrawal", amount, user);
```

```java
        // Log the transaction without checking for duplicates
        fileManager.logTransaction(new Transaction(user.getUserID(), accountNumber,
"Withdrawal", amount));

        // Update the balance in the file
        fileManager.updateAccountBalance(user.getUserID(), accountNumber, balance);

        return true; // Indicate success
    }

    // Log a transaction to history only
    // Creates a transaction object and adds it to the transaction history
    protected void logTransaction(String type, double amount, User user) {
        Transaction transaction = new Transaction(user.getUserID(), accountNumber, type,
amount);
        transactionHistory.add(transaction);
    }

    // Set balance method
    // Validates that the new balance is not negative before updating
    public void setBalance(double newBalance) {
        if (newBalance < 0) {
            throw new IllegalArgumentException("Balance cannot be negative.");
        }
        this.balance = newBalance;
    }

    // Getters
    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public List<Transaction> getTransactionHistory() {
        return transactionHistory;
    }

    // Abstract method for subclasses to define account type
    public abstract String getAccountType();
}
```

CHECKING ACCOUNT CLASS

The CheckingAccount class extends the Account class, adding specific functionality for checking accounts, including an overdraft limit. It initializes the account with a given overdraft limit and allows withdrawals that can exceed the balance as long as they stay within this limit. The withdrawal method checks the validity of the amount, updates the balance, logs the transaction, and records it in the user's transaction history.

```
import java.time.LocalDateTime;

public class CheckingAccount extends Account {
    private double overdraftLimit;

    // Constructor to initialize a CheckingAccount object.
    // It requires an account number, an initial balance, and an overdraft limit.
    public CheckingAccount(String accountNumber, double initialBalance, double overdraftLimit)
{
        super(accountNumber, initialBalance);
        this.overdraftLimit = overdraftLimit;
    }

     // Method to withdraw a specified amount from the account.
    // It checks if the withdrawal amount is valid and if it exceeds the available balance
including the overdraft limit.
    public boolean withdraw(double amount, User user) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive.");
        }

        double availableBalance = balance + overdraftLimit;
        if (amount > availableBalance) {
            throw new IllegalArgumentException(String.format(
                "Withdrawal amount of %.2f exceeds available balance of %.2f (including
overdraft limit of %.2f).",
                amount, balance, overdraftLimit));
        }

        balance -= amount;

        // Create a transaction and log it
        Transaction transaction = new Transaction(user.getUserID(), accountNumber,
"Withdrawal", amount, LocalDateTime.now()); // Include timestamp
        user.addTransaction(transaction); // Add to user's transaction history
```

```java
        FileManager fileManager = new FileManager();
        fileManager.logTransaction(transaction); // Log transaction to file

        return true;
    }

    @Override
    public String getAccountType() {
        return "Checking Account";
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    public void setOverdraftLimit(double overdraftLimit) {
        this.overdraftLimit = overdraftLimit;
    }
}
```

SAVINGS ACCOUNT CLASS

The SavingsAccount class extends the Account class, adding functionality for managing a savings account with an interest rate. It includes a method to apply interest to the account balance and overrides the method to specify the account type as Savings Account.

```java
public class SavingsAccount extends Account {
    private double interestRate;

    // Constructor for SavingsAccount
    public SavingsAccount(String accountNumber, double initialBalance, double interestRate) {
        super(accountNumber, initialBalance);
        this.interestRate = interestRate;
    }

    // Apply interest to the balance
    public void applyInterest(User user, FileManager fileManager) {
        double interest = getBalance() * (interestRate / 100);
        super.deposit(interest, user, fileManager);
    }

    @Override
    public String getAccountType() {
        return "Savings Account";
    }
```

```
    public double getInterestRate() {
        return interestRate;
    }

    public void setInterestRate(double interestRate) {
        this.interestRate = interestRate;
    }
}
```

USER CLASS

The User class represents a bank user, encapsulating their personal details, account information, and transaction history. It allows for managing user attributes such as name, user ID, and PIN, as well as handling multiple accounts and their lock status. The class includes methods to add accounts, check for existing accounts, manage transactions, and retrieve user information, facilitating interactions with the banking system.

```
import java.util.ArrayList;
import java.util.List;

public class User {
    private String name;
    private String userID;
    private String pin;
    private List<Account> accounts;
    private boolean isAccountLocked;
    private List<Transaction> transactionHistory;

    public User(String name, String userID, String pin) {
        this.name = name;
        this.userID = userID;
        this.pin = pin;
        this.accounts = new ArrayList<>();
        this.isAccountLocked = false; // Default account status is unlocked
        this.transactionHistory = new ArrayList<>();
    }

    public String getName() {
        return name;
```

```java
    }

    public String getUserID() {
        return userID;
    }

    public String getPin() {
        return pin;
    }

    public void setPin(String pin) {
        this.pin = pin;
    }

    // Returns the list of accounts associated with the user
    public List<Account> getAccounts() {
        return accounts;
    }

    // Returns the account lock status
    public boolean isAccountLocked() {
        return isAccountLocked;
    }

    // Method to lock or unlock the account
    public void setAccountLocked(boolean accountLocked) {
        isAccountLocked = accountLocked;
    }

    // Adds a new account to the user's account list
    public void addAccount(Account account) {
        accounts.add(account);
    }

    // Check if the user has any accounts
    public boolean hasAccounts() {
        return !accounts.isEmpty();
    }

    // Add transaction to history
    public void addTransaction(Transaction transaction) {
        transactionHistory.add(transaction);
    }

    // Returns transaction history
```

```
    public List<Transaction> getTransactionHistory() {
        return transactionHistory;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append(name).append(",").append(userID).append(",").append(pin); // Include PIN as
plain text
        for (Account account : accounts) {
            sb.append(",").append(account.getAccountNumber())
              .append(",").append(account.getAccountType())
              .append(",").append(account.getBalance());
        }
        return sb.toString();
    }
}
```

TRANSACTION CLASS

The Transaction class models a financial transaction associated with a user and an account, capturing details such as user ID, account number, transaction type, amount, and timestamp. It provides constructors for initializing transactions, getter methods for accessing transaction attributes, and a formatted string representation for display.

```
import java.time.LocalDateTime;

public class Transaction {
    private String userID;
    private String accountNumber;
    private String type; // "Deposit", "Withdrawal", "Interest", etc.
    private double amount;
    private LocalDateTime timestamp;

    // Constructor with timestamp
    public Transaction(String userID, String accountNumber, String type, double amount,
LocalDateTime timestamp) {
        if (amount < 0) {
            throw new IllegalArgumentException("Amount must be non-negative.");
        }
        this.userID = userID;
        this.accountNumber = accountNumber;
        this.type = type;
        this.amount = amount;
        this.timestamp = timestamp;
```

```java
    }

    // Constructor that sets the timestamp to now
    public Transaction(String userID, String accountNumber, String type, double amount) {
        this(userID, accountNumber, type, amount, LocalDateTime.now());
    }

    // Getters
    public String getUserID() {
        return userID;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getType() {
        return type;
    }

    public double getAmount() {
        return amount;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }

    // Override toString for better representation
    @Override
    public String toString() {
        return String.format("User ID: %s | Account: %s | Type: %s | Amount: %.2f | Date: %s",
                userID, accountNumber, type, amount, timestamp);
    }
}
```

FILE MANAGER CLASS

It provides methods to save and load user information from a file, log transactions, retrieve transaction reports for specific users, and update account balances and user passwords. It ensures data integrity by checking for duplicate transactions and formats user and transaction data for storage.

```java
import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
```

```java
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

public class FileManager {
    private static final String FILE_PATH = "users.txt";
    private static final String TRANSACTIONS_FILE = "transactions.txt";
    private Set<String> loggedTransactions = new HashSet<>(); // Track logged transactions

    // Save a list of users to the file
    public void saveUsers(List<User> users) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_PATH))) {
            // Convert user to string and write to file
            for (User user : users) {
                writer.write(userToString(user));
                writer.newLine();
            }
        } catch (IOException e) {
            System.err.println("Error saving users: " + e.getMessage());
        }
    }

    // Load users from the file
    public List<User> loadUsers() {
        List<User> users = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(FILE_PATH))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split(",");
                if (parts.length < 3) {
                    System.out.println("Skipping invalid line: " + line);
                    continue;
                }
                // Extract user information and create User object
                String name = parts[0].trim();
                String userID = parts[1].trim();
                String pin = parts[2].trim();
                User user = new User(name, userID, pin);

                 // Load user accounts
                for (int i = 3; i < parts.length; i += 3) {
                    if (i + 2 >= parts.length) {
                        System.out.println("Skipping incomplete account data for user: " +
userID);
```

```
                        break;
                    }
                    // Create accounts based on account type
                    String accountNumber = parts[i].trim();
                    String accountType = parts[i + 1].trim();
                    double balance = Double.parseDouble(parts[i + 2].trim());

                    Account account;
                    if (accountType.equalsIgnoreCase("Checking Account")) {
                        account = new CheckingAccount(accountNumber, balance, 100);
                    } else {
                        account = new SavingsAccount(accountNumber, balance, 1.5);
                    }
                    user.addAccount(account);
                }
                users.add(user);
            }
        }
        // Handle file read errors
        catch (IOException e) {
            System.err.println("Error loading users: " + e.getMessage());
        }
        // Handle balance parsing errors
        catch (NumberFormatException e) {
            System.err.println("Error parsing balance: " + e.getMessage());
        }
        return users;
    }

    // Log a transaction to the transaction file
    public void logTransaction(Transaction transaction) {
        String transactionKey = transaction.getUserID() + transaction.getAccountNumber() +
transaction.getType() + transaction.getAmount();
        // Check if the transaction has already been logged
        if (!loggedTransactions.contains(transactionKey)) {
            try (BufferedWriter writer = new BufferedWriter(new FileWriter(TRANSACTIONS_FILE,
true))) {
                String logEntry = String.format("%s, %s, %s, %.2f, %s%n",
                        transaction.getUserID(),
                        transaction.getAccountNumber(),
                        transaction.getType(),
                        transaction.getAmount(),
                        transaction.getTimestamp().format(DateTimeFormatter.ofPattern("yyyy-
MM-dd HH:mm:ss")));
                writer.write(logEntry);
```

```
                loggedTransactions.add(transactionKey); // Mark as logged
            } catch (IOException e) {
                System.err.println("Error logging transaction: " + e.getMessage());
            }
        }
    }

    // Get a user's transaction report  based on their user ID.
    public List<Transaction> getUserTransactionReport(String userID) {
        List<Transaction> userTransactions = new ArrayList<>();
        List<User> users = loadUsers();

        for (User user : users) {
            if (user.getUserID().equals(userID)) {
                userTransactions.addAll(user.getTransactionHistory());
                break;
            }
        }

        // Load transactions from the transaction file
        try (BufferedReader reader = new BufferedReader(new FileReader(TRANSACTIONS_FILE))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.split(",\\s*");
                if (parts.length < 5) continue;
                // Create Transaction object from log entry
                String transactionUserID = parts[0].trim();
                String accountNumber = parts[1].trim();
                String type = parts[2].trim();
                double amount = Double.parseDouble(parts[3].trim());
                LocalDateTime timestamp = LocalDateTime.parse(parts[4].trim(),
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));

                if (transactionUserID.equals(userID)) {
                    userTransactions.add(new Transaction(transactionUserID, accountNumber,
type, amount, timestamp));
                }
            }
        } catch (IOException e) {
            System.err.println("Error reading transaction report: " + e.getMessage());
        }

        return userTransactions;
    }
```

```java
    // Update the account balance
    public void updateAccountBalance(String userID, String accountNumber, double newBalance) {
        List<User> users = loadUsers();
        for (User user : users) {
            if (user.getUserID().equals(userID)) {
                for (Account account : user.getAccounts()) {
                    if (account.getAccountNumber().equals(accountNumber)) {
                        account.setBalance(newBalance);
                        break;
                    }
                }
                // Exit the outer loop after finding the user
                break;
            }
        }
        saveUsers(users); // Save changes back to file
    }

    // Update the user's password
    public void updateUserPassword(String userID, String newPassword) {
        List<User> users = loadUsers();
        for (User user : users) {
            if (user.getUserID().equals(userID)) {
                user.setPin(newPassword);
                break;
            }
        }
        saveUsers(users); // Save changes back to file
    }

    // Convert a User object to a string for saving to file
    private String userToString(User user) {
        StringBuilder sb = new StringBuilder();
        sb.append(user.getName()).append(",")
          .append(user.getUserID()).append(",")
          .append(user.getPin());

        // Append account details to the string
        for (Account account : user.getAccounts()) {
            sb.append(",").append(account.getAccountNumber())
              .append(",").append(account.getAccountType())
              .append(",").append(account.getBalance());
        }
        // Return the formatted string
        return sb.toString();
```

```
    }
}
```

ATM CLASS

The ATM class implements a graphical user interface for an ATM system, allowing users to log in, register accounts, and perform banking operations such as deposits, withdrawals, and transfers. It manages user sessions, interacts with the FileManager for data persistence, and displays information and prompts using dialog boxes.

```
import java.awt.*;
import java.util.ArrayList;
import java.util.List;
import javax.swing.*;

// Main class representing the ATM system
public class ATM {
    private User currentUser;
    private List<User> users;
    private FileManager fileManager;

    // Constructor to initialize the ATM system
    public ATM() {
        users = new ArrayList<>();
        fileManager = new FileManager();
        users = fileManager.loadUsers();
    }

    // Method to start the ATM system
    public void start() {
        while (true) {
            String[] options = {"Login", "Register", "Exit"};
            int choice = JOptionPane.showOptionDialog(null, "Welcome to the ATM System",
"ATM",
                JOptionPane.DEFAULT_OPTION, JOptionPane.INFORMATION_MESSAGE, null,
                options, options[0]);
            if (choice == 0) {
                login();
            } else if (choice == 1) {
                registerUser();
            } else if (choice == 2) {
                break;
            }
        }
    }
```

```java
    // Method for user login
    private void login() {
        JTextField userIDField = new JTextField(20);
        JPasswordField pinField = new JPasswordField(20);
        Object[] message = {
            "User ID:", userIDField,
            "PIN:", pinField
        };

        // Show confirmation dialog for login
        int option = JOptionPane.showConfirmDialog(null, message, "Login",
JOptionPane.OK_CANCEL_OPTION);
        if (option == JOptionPane.OK_OPTION) {
            String userID = userIDField.getText();
            String pin = new String(pinField.getPassword());
            for (User user : users) {
                if (user.getUserID().equals(userID) && user.getPin().equals(pin)) {
                    currentUser = user;
                    showFunctionalityPage();
                    return;
                }
            }
            JOptionPane.showMessageDialog(null, "Invalid User ID or PIN.");
        }
    }

    // Method to register a new user
    private void registerUser() {
        JPanel panel = new JPanel(new GridLayout(0, 1));
        JTextField nameField = new JTextField(20);
        JTextField userIDField = new JTextField(20);
        JPasswordField pinField = new JPasswordField(20);
        String[] accountTypes = {"Checking Account", "Savings Account"};
        JComboBox<String> accountTypeComboBox = new JComboBox<>(accountTypes);
        JTextField initialBalanceField = new JTextField(20);

        // Add input fields to the panel
        panel.add(new JLabel("Enter your name:"));
        panel.add(nameField);
        panel.add(new JLabel("Enter a new User ID:"));
        panel.add(userIDField);
        panel.add(new JLabel("Enter your PIN:"));
        panel.add(pinField);
        panel.add(new JLabel("Select Account Type:"));
```

```
        panel.add(accountTypeComboBox);
        panel.add(new JLabel("Enter Initial Balance:"));
        panel.add(initialBalanceField);

        // Show confirmation dialog for registration
        int result = JOptionPane.showConfirmDialog(null, panel, "Register",
JOptionPane.OK_CANCEL_OPTION);
        if (result == JOptionPane.OK_OPTION) {
            String name = nameField.getText();
            String userID = userIDField.getText();
            String pin = new String(pinField.getPassword());
            String accountType = (String) accountTypeComboBox.getSelectedItem();
            String initialBalanceString = initialBalanceField.getText();

            if (userID.isEmpty() || pin.isEmpty() || name.isEmpty() ||
initialBalanceString.isEmpty()) {
                JOptionPane.showMessageDialog(null, "All fields must be filled.");
                return;
            }

            // Validate initial balance
            double initialBalance;
            try {
                initialBalance = Double.parseDouble(initialBalanceString);
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(null, "Initial balance must be a valid
number.");
                return;
            }

            // Check for existing User ID
            for (User user : users) {
                if (user.getUserID().equals(userID)) {
                    JOptionPane.showMessageDialog(null, "User ID already exists. Please choose
another.");
                    return;
                }
            }

            // Create a new account based on the selected type
            Account newAccount;
            String accountNumber = "ACC" + System.currentTimeMillis(); // Unique account
number based on timestamp
            if (accountType.equals("Checking Account")) {
                newAccount = new CheckingAccount(accountNumber, initialBalance, 100);
```

```
            } else {
                newAccount = new SavingsAccount(accountNumber, initialBalance, 1.5);
            }

            // Create a new user with the account
            User newUser = new User(name, userID, pin);
            newUser.addAccount(newAccount);
            users.add(newUser);
            fileManager.saveUsers(users);
            JOptionPane.showMessageDialog(null, "Registration successful! You can now log
in.");
        }
    }

    // Method to show the functionality page after login
    private void showFunctionalityPage() {
        if (currentUser == null || !currentUser.hasAccounts()) {
            JOptionPane.showMessageDialog(null, "No accounts available.");
            return;
        }

        // Account selection
        String[] accountOptions = currentUser.getAccounts().stream()
                .map(Account::getAccountNumber)
                .toArray(String[]::new);
        String selectedAccountNumber = (String) JOptionPane.showInputDialog(null,
                "Select an account", "ATM System",
                JOptionPane.PLAIN_MESSAGE, null,
                accountOptions, accountOptions[0]);

        if (selectedAccountNumber == null) return;

        Account selectedAccount = currentUser.getAccounts().stream()
                .filter(account -> account.getAccountNumber().equals(selectedAccountNumber))
                .findFirst()
                .orElse(null);

        if (selectedAccount == null) {
            JOptionPane.showMessageDialog(null, "Selected account not found.");
            return;
        }

        // Display account information
        String accountInfo = "Current Account:\n" +
                             "Account Number: " + selectedAccount.getAccountNumber() + "\n" +
```

```
                                "Account Type: " + selectedAccount.getAccountType() + "\n" +
                                "Balance: " + selectedAccount.getBalance();

        // Create panel for account operation
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        JLabel infoLabel = new JLabel("<html>" + accountInfo.replace("\n", "<br>") +
"</html>");
        infoLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

        JButton depositButton = new JButton("Deposit");
        depositButton.addActionListener(e -> deposit(selectedAccount));
        JButton withdrawButton = new JButton("Withdraw");
        withdrawButton.addActionListener(e -> withdraw(selectedAccount));
        JButton transferButton = new JButton("Transfer");
        transferButton.addActionListener(e -> transfer(selectedAccount));
        JButton logoutButton = new JButton("Log Out");

        logoutButton.addActionListener(e -> logout());

        panel.add(infoLabel);
        panel.add(Box.createRigidArea(new Dimension(0, 20))); // Space between info and
buttons
        panel.add(depositButton);
        panel.add(withdrawButton);
        panel.add(transferButton);
        panel.add(logoutButton);

        JOptionPane.showMessageDialog(null, panel, "ATM System", JOptionPane.PLAIN_MESSAGE);
    }

    // Method to deposit money into an account
    private void deposit(Account account) {
        String input = JOptionPane.showInputDialog(null, "Enter amount to deposit into account
" + account.getAccountNumber() + ":");
        if (input == null || input.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Deposit amount cannot be empty.");
            return;
        }
        try {
            double amount = Double.parseDouble(input);
            if (amount <= 0) {
                JOptionPane.showMessageDialog(null, "Deposit amount must be greater than
zero.");
                return;
```

```
            }
            account.deposit(amount, currentUser, fileManager); // Updated call
            JOptionPane.showMessageDialog(null, "Deposit successful! New balance: " +
account.getBalance());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Invalid amount entered. Please enter a
numeric value.");
        }
    }

    // Method to withdraw money from an account
    private void withdraw(Account account) {
        String input = JOptionPane.showInputDialog(null, "Enter amount to withdraw from
account " + account.getAccountNumber() + ":");
        if (input == null || input.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Withdrawal amount cannot be empty.");
            return;
        }
        try {
            double amount = Double.parseDouble(input);
            if (amount <= 0) {
                JOptionPane.showMessageDialog(null, "Withdrawal amount must be greater than
zero.");
                return;
            }
            account.withdraw(amount, currentUser, fileManager);
            JOptionPane.showMessageDialog(null, "Withdrawal successful! New balance: " +
account.getBalance());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Invalid amount entered. Please enter a
numeric value.");
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(null, e.getMessage());
        }
    }

    // Method to transfer money between accounts
    private void transfer(Account fromAccount) {
        if (currentUser.getAccounts().size() < 2) {
            JOptionPane.showMessageDialog(null, "You need at least two accounts to
transfer.");
            return;
        }

        String toAccountNumber = (String) JOptionPane.showInputDialog(null,
```

```
            "Transfer from account " + fromAccount.getAccountNumber() + "\n" +
            "Select account to transfer to:",
            "Transfer",
            JOptionPane.PLAIN_MESSAGE,
            null,
            currentUser.getAccounts().stream()
                .filter(account -> !account.equals(fromAccount)) // Exclude the fromAccount
                .map(Account::getAccountNumber)
                .toArray(String[]::new),
            null);

        if (toAccountNumber == null) return; // User canceled selection

        Account toAccount = currentUser.getAccounts().stream()
                .filter(account -> account.getAccountNumber().equals(toAccountNumber))
                .findFirst()
                .orElse(null);

        String input = JOptionPane.showInputDialog(null, "Enter amount to transfer from
account " + fromAccount.getAccountNumber() + " to account " + toAccountNumber + ":");
        try {
            double amount = Double.parseDouble(input);
            if (amount <= 0) {
                JOptionPane.showMessageDialog(null, "Transfer amount must be greater than
zero.");
                return;
            }
            fromAccount.withdraw(amount, currentUser, fileManager); // Updated call
            toAccount.deposit(amount, currentUser, fileManager); // Updated call
            JOptionPane.showMessageDialog(null, "Transfer successful!\nNew balance for " +
fromAccount.getAccountNumber() + ": " + fromAccount.getBalance() + "\nNew balance for " +
toAccountNumber + ": " + toAccount.getBalance());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Invalid amount entered. Please enter a
numeric value.");
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(null, e.getMessage());
        }
    }

    // Method to log out the current user
    private void logout() {
        if (JOptionPane.showConfirmDialog(null, "Are you sure you want to log out?", "Log
Out", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
            currentUser = null;
```

```
            JOptionPane.showMessageDialog(null, "You have been logged out.");
        }
    }

    // Main method to run the ATM application
    public static void main(String[] args) {
        ATM atm = new ATM();
        atm.start();
    }
}
```

ADMIN CLASS

It allows the admin to reset user PINs and unlock accounts by prompting for a user's PIN and presenting options for account management. Additionally, the class can generate detailed user reports, displaying account balances and transaction histories, thereby facilitating oversight and account maintenance.

```
import java.awt.*;
import java.util.List;
import javax.swing.*;

public class Admin {
    private List<User> users;
    private FileManager fileManager;

    // Constructor to initialize the Admin with a list of users
    public Admin(List<User> users) {
        this.users = users;
        this.fileManager = new FileManager();
    }

    // Manage user accounts
    // Prompts for a user PIN and manages the selected user if found
    public void manageAccounts() {
        JPasswordField pinField = new JPasswordField();
        int option = JOptionPane.showConfirmDialog(null, pinField,
                "Enter User PIN:", JOptionPane.OK_CANCEL_OPTION);
        if (option == JOptionPane.OK_OPTION) {
            String enteredPin = new String(pinField.getPassword());
            User selectedUser = findUserByPin(enteredPin);
            if (selectedUser != null) {
                manageSelectedUser(selectedUser);
            } else {
```

```java
                JOptionPane.showMessageDialog(null, "No user found with that PIN.");
            }
        } else {
            JOptionPane.showMessageDialog(null, "No action taken.");
        }
    }

    // Manage actions for the selected user
    // Prompts the admin to choose between resetting the PIN or unlocking the account
    public void manageSelectedUser(User user) {
        String action;
        do {
            action = JOptionPane.showInputDialog(null,
                    "Choose an action:\n1. Reset PIN\n2. Unlock Account\nEnter 1 or 2:");
        } while (!"1".equals(action) && !"2".equals(action));

        if ("1".equals(action)) {
            resetPin(user);
        } else if ("2".equals(action)) {
            unlockAccount(user);
        }
    }

    // Find user by PIN
    // Returns the user if found, otherwise returns null
    private User findUserByPin(String pin) {
        for (User user : users) {
            if (user.getPin().equals(pin)) {
                return user;
            }
        }
        return null;
    }

    // Reset the user's PIN
    // Prompts the admin for a new PIN and confirms it
    private void resetPin(User user) {
        JPanel panel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(10, 10, 10, 10);

        // New PIN field
        gbc.gridx = 0; gbc.gridy = 0;
        panel.add(new JLabel("Enter New PIN:"), gbc);
```

```java
        JPasswordField newPinField = new JPasswordField(20);
        gbc.gridx = 1; panel.add(newPinField, gbc);

        // Confirm PIN field
        gbc.gridx = 0; gbc.gridy = 1;
        panel.add(new JLabel("Confirm New PIN:"), gbc);
        JPasswordField confirmPinField = new JPasswordField(20);
        gbc.gridx = 1; panel.add(confirmPinField, gbc);

        // Set the dialog
        int result = JOptionPane.showConfirmDialog(null, panel, "Reset PIN",
                JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);
        if (result == JOptionPane.OK_OPTION) {
            String newPin = new String(newPinField.getPassword()).trim();
            String confirmPin = new String(confirmPinField.getPassword()).trim();

            if (!newPin.equals(confirmPin)) {
                JOptionPane.showMessageDialog(null, "PINs do not match. Please try again.");
                return;
            }

            user.setPin(newPin);
            JOptionPane.showMessageDialog(null, "PIN reset successfully.");
        }
    }

    // Unlock the user's account
    // Sets the account status to unlocked and saves the updated user list
    private void unlockAccount(User user) {
        user.setAccountLocked(false);
        JOptionPane.showMessageDialog(null,
                "Account unlocked for User ID: " + user.getUserID());
        fileManager.saveUsers(users);
    }

    // Generate user report
    public void generateUserReport(User user) {
        StringBuilder report = new StringBuilder();
        report.append("User Report for ").append(user.getName()).append(" (")
              .append(user.getUserID()).append("):\n");
        for (Account account : user.getAccounts()) {
            report.append("Account Number: ").append(account.getAccountNumber())
                  .append(", Balance: ").append(account.getBalance()).append("\n");
        }
        report.append("Transaction History:\n");
```

```
        for (Transaction transaction : user.getTransactionHistory()) {
            report.append(transaction.toString()).append("\n");
        }
        JOptionPane.showMessageDialog(null, report.toString());
    }
}
```